Tab 1

Day 1 (Module A1) Competition Playbook — Framework, Task Split, and Vulnerability Assessment

Assessing Security: an Apache website on a Linux server, scoped to that service unless something explicitly ties into access control. Not a pentest — an investigation of an existing configuration, no internet, short window, one shared deliverable (Executive Summary + Table 1 in Appendix 1).

Team: Earl (OS-level administration and security — Rocky Linux/RHEL, Windows Server) & Ryle (network infrastructure, routing, firewall)

Read this first: a scope clue worth building your whole approach around

Test Project A1 restricts you to *"the Apache installation and website configuration, not other machines or configurations **unless explicitly referenced for access control validation**."* That caveat wouldn't need to exist if the "authorised group" access control were purely local to the Linux box — a local Unix group needs no such carve-out. The fact that the Organizer felt it necessary to write that exception strongly suggests the access control is **AD/LDAP-backed**, not local.

That also lines up with something from your Day 2 prep: Table 2's DMZ→Servers firewall rule is scoped to *"AD DS required ports (minimum)... only what is required for auth/policy"* — a rule that only makes sense if LinuxWeb genuinely talks to WinSRV1 for authentication. Between the two documents, there are now two independent signals pointing at the same thing: **the Linux web server probably does LDAP-based auth against WinSRV1, even without a full domain join.** Worth testing for this specifically on Day 1 rather than assuming it's a local group — and worth revisiting the "domain join is optional" call in your LinuxWeb hardening guide once you've confirmed it.

---

1. The Framework

A full CIS Apache HTTP Server Benchmark won't work here — it runs 100+ controls, and you have no internet to pull it during the module anyway. What actually fits the constraints (config review, not exploitation; short window; two vulnerabilities out of everything you find; non-technical writeup) is a **distilled CIS/OWASP-derived checklist**, run as a fixed chronological sequence, memorized or brought in as personal notes before the module starts, since Instructions to the Competitor explicitly says internet access isn't available.

The Blueprint

Phase	Goal	Suggested share of your time budget
0. Baseline Verify	Confirm static IPs/hostnames match documentation, per Instructions to the Competitor	5%
1. Parallel Recon	Both of you gather raw evidence simultaneously (see Section 2)	20%
2. Config Deep-Dive	Read every relevant Apache/system config file line by line	30%
3. Access Control Validation	Prove or disprove the authorised-group restriction actually works	20%
4. Score & Select	Rate everything found, pick the two most critical (Section 4)	10%
5. Write-Up	Executive summary + Table 1, evidence attached	15%

Scale these percentages to whatever the actual module duration turns out to be — the point is the *order* and the *parallel structure* in Phase 1, not the exact minutes.

Prep before competition day: since you can't look anything up live, build yourselves a one-page offline cheat sheet of Apache version → known-CVE thresholds (e.g. the 2.4.49/2.4.50 path traversal CVEs, older mod_ssl/mod_proxy issues) so a version banner alone can trigger recognition without a lookup.

---

2. Task Delegation

Given your split — Earl (OS-level, Rocky/Windows) and Ryle (network/firewall) — Module A1's scope is narrower for the network side than Module A2 will be, but there's still real parallel work, not idle time.

Phase 1: Parallel Recon (run simultaneously, not sequentially)

Earl (OS-level)	Ryle (Network)
SSH into the Linux server, start reading Apache configs directly	From a separate client, run `nmap -sV -p- <linuxweb-ip>` — full port scan, service/version detection
`apachectl -S` — dump the full vhost/config structure	`curl -I http://<ip>` and `https://<ip>` — capture headers, `Server` banner, redirect behavior
`httpd -M` — list every loaded module	Browser-based walkthrough of the site as an anonymous/unauthenticated visitor — note what's reachable without credentials
Check `/var/log/httpd/access_log` and `error_log` for anything already revealing	`openssl s_client -connect <ip>:443` — protocol version, cipher suite, cert details

Neither of you blocks the other here — you're both generating independent evidence that gets cross-referenced in Phase 2.

Phase 2: Config Deep-Dive (you lead, teammate reviews/cross-checks)

This phase is inherently host-based, so it's mostly your work — but structure it as "you read, teammate verifies the finding independently" rather than one person doing it alone, since the deliverable's evidence needs to hold up to a second look before it goes in Table 1.

Phase 3: Access Control Validation (joint)

Given the AD/LDAP hypothesis above, this is genuinely a two-system problem:

- **Earl:** on LinuxWeb, find and read the auth-related Apache directives (`AuthLDAPURL`, `Require ldap-group`, or equivalent) and any SSSD/LDAP client config
- **Ryle:** on WinSRV1 (their domain is networking, but AD group membership is a quick, joint check either of you can do), confirm which accounts are actually in the authorised group

- **Both:** attempt to access the restricted content as a member and as a non-member, from a client machine — this is the test that either confirms or breaks the access control, and it's exactly the client's stated concern

Phase 4–5: Score, Select, Write

Joint — this shouldn't be one person's writeup of the other's findings. Fill Table 1 together so severity/risk ratings reflect agreement, not just whoever typed first.

---

3. Vulnerability Assessment Playbook

Boundary reminder: every command below is *observation*, not exploitation — read configs, check states, test access with provided credentials. Don't run actual exploit code even if a version match suggests a known CVE; document the exposure and move on. That's the "not a pentest" line the Test Project draws.

A. Information Disclosure - Earl

Check	Command	What you're looking for
Version banner	`curl -I http://<ip>`	`Server:` header revealing Apache version, OS
ServerTokens/Signature	`grep -i "ServerTokens|ServerSignature" /etc/httpd/conf/httpd.conf`	Should be `ServerTokens Prod` / `ServerSignature Off`; anything else leaks detail
Default content left in place	Browse `/`, `/manual`, `/icons`, `/server-status`, `/server-info`	Default Apache pages confirm version, sometimes expose live internal state

mod_status/mod_info exposed	`curl http://<ip>/server-status`	Publicly reachable = real finding, shows live connection/request data to anyone
Directory listing	Browse any directory without an index file	`Options +Indexes` if a bare file listing appears

B. Access Control (the one the client actually asked about) - Earl

Check	Command	What you're looking for
Auth mechanism in use	`grep -riE "AuthType|AuthLDAP|AuthBasic|Require" /etc/httpd/conf/httpd.conf /etc/httpd/conf.d/*.conf`	Basic auth + `.htpasswd`? LDAP-bound `Require ldap-group`?
`.htaccess` actually enforced	`grep -i "AllowOverride" /etc/httpd/conf/httpd.conf` (per directory)	If `AllowOverride None` but the restriction lives in a `.htaccess` file, it's silently **not enforced at all** — this is a classic, high-severity finding
LDAP/AD tie-in	`cat /etc/sssd/sssd.conf` (if present), `id <testaccount>`, `getent group <authorized-group>`	Confirms or disproves the AD-backed hypothesis from the top of this document
Credential test	Attempt login as an authorised member, then as a non-member	Direct proof the control does or doesn't work
Weak/default credentials	Check `.htpasswd` if used: `cat /etc/httpd/.htpasswd` (hashed, but presence/algorithm is informative)	Weak hash algorithm (plain MD5/crypt) is a documentable weakness

C. Transport Security - Ryle

Check	Command	What you're looking for
Protocol/cipher	`openssl s_client -connect <ip>:443 -tls1` then retry with `-tls1_2`	If old TLS 1.0/1.1 negotiates successfully, that's a finding
Cert validity	Same `openssl s_client` output	Self-signed, expired, or hostname-mismatched cert
HTTP available alongside HTTPS	`curl -I http://<ip>`	If sensitive/restricted content is also reachable over plain HTTP, that's a real issue, not just theoretical

D. Filesystem and Process Hygiene - Ryle

Check	Command	What you're looking for
Running user	`ps aux | grep httpd`	Should show `apache`/`www-data` worker processes, not root
Webroot permissions	`ls -la /var/www/html` (or actual docroot)	World-writable files/directories are a tampering risk
Exposed sensitive files	`curl http://<ip>/.git/config`, `/backup.zip`, `/config.php.bak`, etc. — try common leftover filenames	Backup/version-control artifacts left in the docroot
SELinux context sanity	`ls -Z /var/www/html`, `sestatus`	Context mismatches sometimes reveal the docroot was moved/modified outside normal tooling — informative, not itself a vulnerability

E. Firewall/Network Angle - Ryle

Check	Command	What you're looking for
Exposed ports beyond expected	`nmap -sV -p- <ip>`	Anything besides 80/443/22 running is worth documenting
Local firewall state	`sudo firewall-cmd --list-all` on the server itself	Confirms whether the host firewall is actually restricting anything, or if everything's wide open locally regardless of network-level controls

---

4. Scoring Table 1 Defensibly

Use a consistent, explainable rule rather than gut-feel numbers — a grader checking your severity/risk pairing against your own description is a fast way to lose credibility if the numbers don't track the CVSS-style convention:

Severity (0–10)	Risk Label
0.1 – 3.9	Low
4.0 – 6.9	Medium
7.0 – 8.9	High
9.0 – 10.0	Critical

When justifying a number, reason through it out loud in your notes: *what's the impact if exploited* (confidentiality/integrity/availability), *how easy is it* (no auth needed vs. requires valid credentials), *how exposed is it* (public internet-facing vs. requires prior access). A broken access control on the exact restricted functionality the client asked about is very likely your highest-scoring, most defensible pick for one of the two slots — it's directly responsive to the brief, not a tangential finding.

---

5. Documentation Methodology — Every A1 Marking Scheme Row

A1 is worth 15 marks total, split across 8 aspects. Unlike Day 2, most of these aren't "configure X correctly" checkpoints — they're checkpoints on **how well you documented what you found**, which means the writing itself is scored line by line, not just accepted as a container for your findings.

The 8 rows, and what each one actually demands

#	Aspect	Marks	Type	What earns it
1	Submission compliance and scope lock	2	M	1 mark: report submitted within the word limit **with exactly two findings** — not one, not three. 1 mark: every required field complete, correct frozen filename, assessment stayed in scope
2	Finding 1 evidence traceability	2	M	1 mark: evidence source is identifiable and retrievable. 1 mark: the evidence actually supports the stated finding
3	Finding 2 evidence traceability	2	M	Same, for Finding 2
4	Executive summary audience clarity	2	J (0–3)	Clear, accurate, appropriately expressed for a **non-technical** stakeholder
5	Executive summary risk communication	2	J (0–3)	Risk and business impact accurately communicated, priorities understandable
6	Risk prioritization and criticality reasoning	2	J (0–3)	Findings correctly prioritized by real impact/likelihood — top score explicitly requires **distinguishing true critical issues from decoy or compensating-control conditions**
7	Remediation technical appropriateness	2	J (0–3)	The fix is technically correct and addresses the actual root cause

8	Remediation practicality and sequencing	1	J (0–3)	The fix is feasible, minimally disruptive, and well-ordered — separate from whether it's *correct*

Two things worth flagging before you write a word:

- **"Exactly two findings" is itself graded** (row 1). Padding to three "just in case" or only writing up one costs you a mark regardless of quality — pick two and commit.
- **Row 6's decoy-condition language is a real trap to plan for.** Something can look severe (e.g. a scary-sounding exposed page) but actually have a compensating control elsewhere (blocked at the firewall, no sensitive data behind it) that makes it lower real-world risk than something quieter but genuinely exploitable — like a broken access control on the one thing the client explicitly cares about. Before finalizing your two, ask of each candidate: *is anything else in the environment already mitigating this?* The top-scoring answer here isn't "list the scariest-looking things," it's "prove you can tell real risk from a decoy."

Per-Finding Documentation Template

Use this structure for **each** of your two findings — it's built to hit rows 2/3, 6, 7, and 8 directly, not just describe the bug:

1. Vulnerability (Table 1 column 1) — short, specific title.

2. Evidence (feeds rows 2/3 — write this even though Table 1 doesn't have its own column for it; keep it as your backing reference, e.g. in an appendix or evidence folder) — the exact command run, its output, or a screenshot filename, plus when it was captured. "Server signature disclosed" is not evidence; `curl -I http://192.168.1.10` → `Server: Apache/2.4.37 (rocky)` at 09:42, saved as `evidence/finding1-headers.txt` is.

3. Description — what the configuration actually is, in technical terms.

4. Severity (0–10) / Risk label — using the CVSS-aligned table from Section 4.

5. Why is this a problem (feeds row 6) — plain-terms impact, **and** an explicit note on why this beats the decoy candidates you considered. One sentence is enough: *"Unlike [other finding you didn't pick], this is directly reachable with no authentication and touches the exact access-control concern the client raised."*

6. Recommendation (feeds rows 7 and 8 — write it in two parts, not one blended paragraph):

- *Technical fix:* the precise, root-cause-correct change (e.g. "Set `AllowOverride AuthConfig` on the restricted directory so the existing `.htaccess` Require directive is actually enforced" — not just "improve access control")
- *How/when to apply it:* feasibility and sequencing — is it a zero-downtime config reload, or does it need a restart during a maintenance window; does it depend on the other finding being fixed first or can they happen in either order

Executive Summary (feeds rows 4 and 5 — 500 words, non-technical)

- Lead with the answer to the client's actual question — safe to publish, and does access restriction work — in the first two sentences, not buried in paragraph three
- No CVE numbers, no config file names, no command output — describe impact in plain terms ("an outside visitor can currently view content meant to be restricted to a specific group" rather than "`AllowOverride None` defeats the `.htaccess`-based `Require` directive")
- Structure: current state → the two key risks in plain language, each with its business impact stated explicitly (row 5 grades this directly) → what fixing them buys the client → nothing else
- Table 1 carries the technical detail; the summary shouldn't duplicate it, just frame it for someone who will never open Table 1

Before You Submit — Compliance Pass (row 1)

- [ ] Exactly two findings in Table 1, no more, no fewer
- [ ] Executive summary is under 500 words — actually count it, don't estimate
- [ ] Every column in Table 1 is filled for both findings — Vulnerability, Description, Severity, Risk, Why It's a Problem, Recommendation
- [ ] Filename matches the frozen naming convention from Competition Day Instructions exactly — confirm this the moment you have it, since it's not published in the Test Project text itself
- [ ] Nothing in the report strays outside Apache/website scope unless it's the access-control/AD tie-in explicitly permitted by the scope carve-out

Ranking the playbook's checklist by severity if found — highest first, since that's what should drive which two make it into Table 1:

Rank	Finding	Severity	Why it ranks here
1	Non-member successfully accesses restricted content	**9–10, Critical**	Direct, empirical proof the access control fails entirely — exactly the client's stated concern, no interpretation needed
2	`.htaccess` restriction silently not enforced (`AllowOverride None`)	**7–9, High–Critical**	Often *the cause* of #1 — if you find this, #1 is very likely also true
3	Exposed sensitive files (`.git/config`, backups, `config.php.bak`)	**7–9.5, High–Critical**	Can leak credentials or source directly — sometimes the thing that compounds into a worse finding elsewhere
4	Restricted content also reachable over plain HTTP	**7–8, High**	Bypasses the entire HTTPS-restriction assumption; credentials/session data in cleartext
5	Apache running as root	**7–8, High**	Doesn't leak anything by itself, but multiplies the blast radius of every other finding on this list
6	World-writable webroot	**7–8, High**	Tampering/defacement/backdoor-planting risk
7	Weak/default credentials or crackable hash	**5–7, Medium–High**	Real, but requires an extra step (cracking) versus #1's direct proof
8	`mod_status`/`mod_info` publicly exposed	**5–7, Medium–High**	Live internal state, not just a version string — more useful to an attacker than passive banners

9	Unexpected exposed network port	**5–7, Medium–High**	Depends heavily on what the service actually is
10	Directory listing enabled	**4–6, Medium**	Impact is entirely dependent on what's sitting in the listed directory
11	Host firewall wide open locally	**4–6, Medium**	Defense-in-depth gap, not a direct exposure on its own
12	TLS 1.0/1.1 still negotiates	**4–5, Medium**	Exploitable, but needs network position
13	Cert invalid/self-signed/expired	**3–5, Low–Medium**	Mostly a trust/usability problem
14	Version banner disclosed	**2–3, Low** (conditional)	Jumps hard if it matches something on your offline CVE cheat sheet — otherwise minor
15	Default content left in place	**1–3, Low**	Confirms install details, little else
16	`ServerTokens`/`ServerSignature` not hardened	**1–2, Low**	Pure info disclosure
17	SELinux context mismatch	**0–2, Informational**	Not a vulnerability by itself — a drift signal

Two things this ranking should change about how you pick the final two:

#1 and #2 are very likely the same root cause, not two separate findings. If the non-member access succeeds *because* `AllowOverride None` is silently ignoring the `.htaccess` restriction, writing both up as separate Table 1 rows is padding the same issue twice, not two distinct vulnerabilities — pick the one that's more evidence-direct (usually #1, since "I logged in as a non-member and it worked" is harder evidence to argue with than a config-file inference) and let its description mention the `AllowOverride` root cause rather than splitting it.

This table is a prior, not a verdict — it's exactly the decoy trap from Section 5, row 6. An exposed port ranked Medium-High could turn out to be firewalled off entirely (decoy, lower real risk than its rank suggests), while something ranked lower here could turn out to have no compensating control at all in your specific environment. Use this ranking to decide *where to look first*, then let what you actually observe — not the prior ranking — decide the final two.

Tab 2

WorldSkills Cybersecurity: A1 team runbook

Prepared for Ryle and teammate from the supplied WorldSkills Philippines 2026 A1 Test Project and Rev.1.2 marking scheme. Reference checks: 7 September 2026.

Use Day 1 to establish two defensible website findings and submit a clear report. The A1 PDF specifies an investigation of an existing Apache website. It limits other-machine assessment to configurations explicitly referenced for website access-control validation. All VMs already have static IPs. Recommendations are required; general hardening is a Day 2 activity.

The marking scheme calls Day 2 **Module A2**, under subcriterion **B1**. This runbook uses “Day 2 / A2 (B1)” for that work. The workbook's “CIS-Validated” wording concerns its scoring-system import structure; it does not establish compliance with Center for Internet Security controls.

The actual duration, topology, addresses, restricted URL, authorized group, credentials, firewall product, submission filename and evidence-export convention are not provided. Replace every example with organizer-supplied values. No actual vulnerability has been observed in the competition VMs, which were not available for this review. The supplied marking scheme is the planning basis; the issued competition instructions control the final requirements.

1. Follow the marks and scope.

A1 scored aspect	Marks	Completion test
Submission compliance and scope	2	Official template; exactly two findings; executive summary at most 500 words; all fields complete; correct frozen filename; assessment in scope
Finding 1 evidence traceability	2	Evidence source is identifiable/retrievable and materially supports the finding
Finding 2 evidence traceability	2	Same standard, independently satisfied for the second finding
Executive summary clarity	2	A non-technical stakeholder can understand the assessment
Executive summary risk communication	2	Impact and priority are accurate and concise

Risk prioritization and criticality reasoning	2	Likelihood, impact and compensating controls support the ranking
Remediation technical appropriateness	2	Recommendations address the evidenced root cause without creating unnecessary risk
Remediation practicality and sequencing	1	Actions are feasible, ordered and minimally disruptive
Total	**15**	**Verified against the workbook's Max Mark cells**

Source: supplied A1 PDF, physical PDF pages 3–6, and workbook sheet “CIS Marking Scheme Import,” A1 aspects at rows 31–58. Use physical PDF page order because later printed page numbers are inconsistent. Workbook totals are A1 15, A2 55 and A3 30 marks. The 0–3 judgement descriptions are rating anchors, not extra marks to add to the Max Mark column.

Evidence and executive-summary quality account for 8 of the 15 A1 marks. Preserve substantial time for them. A long scanner output does not establish the root cause, prove a finding or communicate business impact by itself.

2. Use a shortened NIST assessment workflow with a small CIS checklist.

Use NIST SP 800-115 as the existing methodological foundation: plan the assessment, examine and test the authorized configuration, analyze findings, and recommend mitigation. Its review techniques include configuration, ruleset and log review. These fit A1 closely.

The sequence below is a custom competition adaptation, not an official NIST or WorldSkills framework. Use CIS Controls as a coverage reminder: Control 4 for secure configuration, Control 6 for access control, and Control 8 for logs. A1 does not require a full enterprise CIS audit.

Gate	Work	Required output before moving on
Scope	Read issued requirements; identify website, identities, permitted clients and freeze rules	One agreed target/test card
Baseline	Record Apache/vhost state, addresses, provider, time and current reachability	Enough context to reproduce every test
Validate	Pair local configuration review with ordinary client access tests	Requirement + observed configuration + observed behavior

Prioritize	Compare confirmed candidates, actual exposure and compensating controls	Exactly two distinct, defensible root causes
Report	Complete both Table 1 rows and a plain-language executive summary	Every claim linked to retrievable evidence
Freeze	Cross-check and export using the issued convention	Readable, complete submission in the correct location

For each candidate, write: **expected behavior → actual behavior → controlling configuration → practical impact → minimum corrective action → verification after the fix.**

3. Split Day 1 by evidence source.

You own the Linux/Apache configuration, authentication-provider interpretation, relevant permissions, server logs and technical accuracy of the recommendations. Your teammate owns client-side access tests, permitted source-location comparisons, HTTP/TLS observations, network-path evidence, the evidence index and final export. Your teammate can draft the executive summary while you finish Table 1. You own the final report file; each person supplies text/evidence for the other to review.

Do not force a Windows or firewall audit into A1 merely to preserve your normal Day 2 roles. If either is explicitly named as an access-control dependency, inspect only the part that establishes the website access outcome.

No A1 duration appears in the supplied files. Use the percentages below for the real duration. The minutes are an illustrative **120-minute rehearsal**, not an official schedule.

Elapsed share	120-minute example	You: OS/Apache	Teammate: clients/path/evidence	Handoff or exit condition
0–10%	0–12 min	Confirm target, access, template and scope	Confirm permitted test clients, source IPs, filename and freeze instructions	Agree on one test card; start report skeleton
10–30%	12–36 min	Inventory service; identify active vhost, includes, document root and auth provider	Check documented DNS/IP/path; test public URL, anonymous restricted access and TLS	Share restricted URL and provider as soon as known

30–50%	36–60 min	Trace group authorization, overrides, relevant file permissions and logs	Test member/non-member matrix and documented alternative access paths	Link request evidence to configuration and logs
50–60%	60–72 min	Recheck root causes and practical fixes	Independently reproduce key claims and examine competing explanations	Select two findings; stop routine discovery
60–80%	72–96 min	Complete Table 1, severity reasoning and recommendation sequence	Draft executive summary; assemble evidence index and files	Complete report draft
80–90%	96–108 min	Review summary accuracy and evidence provenance	Review technical claims, source identity, expected/actual behavior and reproducibility	Resolve only concrete gaps in the two findings
90–100 %	108–120 min	Confirm fields, word count, exactly two rows and final content	Prepare prescribed export, inspect it and stage submission	Ready before freeze; follow announced export procedure

A useful hard stop is **60% elapsed: choose findings; 80%: complete draft; 90%: package**. Reopen discovery after 60% only when a selected finding fails validation or a materially stronger, already-evidenced issue appears.

Use these coordination rules:

- One owner per device and one owner for the report file. Avoid simultaneous configuration changes.
- Send an initial target card immediately: website FQDN, destination IP/port, restricted path, authentication method, group identity/provider, test-account labels and source-location labels.
- Record each task as TODO, RUNNING, BLOCKED or DONE. DONE requires an evidence reference and an expected-versus-actual result.
- After three minutes blocked, send the peer the source IP, destination IP, protocol/port, timestamp, exact symptom and evidence ID. Work on an independent task while the owner investigates.
- Limit routine status exchanges to about one minute at the scheduled gates.

- Preserve the initial state. Make only discrepancy corrections explicitly permitted by the issued task, recording before/after state and the reason. Keep general changes for Day 2.

4. Prepare the target card and evidence register.

Field	Fill from issued instructions or observed state
Website identity	FQDN, IP, port, scheme, required public and restricted URLs
Requirement	Exact group allowed to use each restricted function
Identity provider	Apache file/group file, LDAP/AD, another provider, or application session
Test identities	Approved member and valid non-member; anonymous baseline
Client vantage	Source hostname/IP, zone and whether access should be possible from that location
Supporting systems	Only explicitly permitted dependencies
Time	Client/server timestamp, timezone and observed offset
Submission	Official template, filename, location, export format and freeze time

Use the organizer's evidence naming and handling rules. The following IDs are internal examples, not prescribed filenames:

Evidence ID	Suggested content
E01	Target card and initial service/vhost context
E02	Numbered, relevant configuration excerpt and original source path
E03	Provider/group evidence for the member and non-member
E04	Anonymous request: headers, relevant body and source/time
E05	Authorized-member request and result
E06	Valid non-member request and result
E07	TLS/HTTP behavior or another independently evidenced candidate
E08	Corresponding server or permitted firewall log records

Each evidence-index entry needs: ID, original host/path or URL, command/test, source client/IP, destination, timestamp/timezone, account label, expected outcome, actual outcome and the report finding it supports. Include line references for configuration excerpts and event/packet references where available. Keep passwords, private keys and session tokens out of report screenshots. Handle any required raw sensitive evidence according to the organizer's instructions.

The A1 task assumes no Internet connectivity. Practice using the permitted installed tools and local help. Do not spend module time downloading scanners or waiting for online vulnerability feeds. Whether personal notes, prepared scripts or extra tools may be brought in is an organizer rule; this runbook does not assume they are permitted.

5. Inspect Rocky/RHEL and Apache locally.

Run the snippets individually from the appropriate authorized machine. They query configuration or generate evidence files; they are not a deployment script. Normal requests and administrative queries may produce logs. Replace example paths, hostnames and ports first.

Start with the platform and service state:

```
date -u +%FT%TZ
hostnamectl
cat /etc/os-release
ip -br address
ip route
rpm -q httpd mod_ssl openssl
sudo systemctl status httpd --no-pager
sudo systemctl cat httpd
sudo httpd -V
sudo httpd -t
sudo httpd -S
sudo httpd -M
sudo ss -lntp
```

Interpret the Apache listener against the documented endpoints. `httpd -t` checks syntax, `-S` shows virtual-host parsing and `-M` lists modules. Check the systemd unit for an alternative `-f` configuration path or wrapper: use the same configuration arguments as the running service for inspection. Successful syntax checks do not prove correct authorization. These invocations parse files on disk; corroborate their relevance with the running service and live responses. [Apache httpd options]

Locate the relevant directives. If `rg` is absent, use `grep -RniE` with the same pattern. Inspect the full enclosing configuration blocks after this search; the matches alone are not the effective policy.

```
sudo rg -n -i \
'Include|VirtualHost|ServerName|ServerAlias|DocumentRoot|Alias|Directory|Location|Files|AuthType|AuthBasicProvider|AuthUserFile|AuthGroupFile|AuthLDAPURL|Require|AuthMerging|AllowOverride|Options|SSLProtocol|SSLCipherSuite|SSLCertificate|Redirect|RewriteRule|CustomLog|ErrorLog' \
  /etc/httpd/conf /etc/httpd/conf.d
```

Follow active `Include`/`IncludeOptional` paths, including paths outside these directories. Identify the exact vhost for the tested Host header and TLS SNI. Account for `<Directory>`, `<Files>`, `<Location>`, conditional sections and `.htaccess`; their ordering and merging can change which rule wins. Inspect a selected file with `sudo nl -ba /actual/config/file.conf`. A matching directive in an unused backup configuration proves nothing about the running website. Apache configuration sections

Prioritize these questions:

Review question	Interpretation
Does authorization enforce the required group?	`Require valid-user` accepts the provider's valid users; establish whether that population is broader than the required group
Could another authorization clause allow the same request?	Examine `<RequireAny>`, `<RequireAll>`, `AuthMerging` and applicable overrides
Is `Require all granted` relevant to the restricted resource?	It may be correct for public content; prove its effective scope
Does `.htaccess` contain the intended restriction but get ignored?	Establish the relevant `AllowOverride` value and test the affected path
Does HTTP or a documented alias reach the restricted content differently?	Compare only issued or configuration-identified, in-scope paths
Can website-delivered files expose authentication material or private configuration?	Establish actual serving rules and reachability, not just local presence

`Require valid-user` is not automatically a vulnerability: a provider containing only authorized members may satisfy the requirement. Conversely, a broad OR condition can defeat a group rule. Select a finding only after tracing the effective provider and behavior. Apache authentication, authorization containers and merging

Inspect only the website's actual paths:

```
# Examples only: replace using the active vhost configuration.
DOCROOT='/actual/document/root'
PROTECTED_FILE='/actual/document/root/restricted/index.html'
AUTH_FILE='/actual/authentication/file'

sudo find "$DOCROOT" -xdev -type f -name .htaccess -print
sudo namei -l "$PROTECTED_FILE"
sudo getfacl -p "$DOCROOT" "$PROTECTED_FILE" "$AUTH_FILE"
sudo ls -ldZ "$DOCROOT" "$PROTECTED_FILE" "$AUTH_FILE"
getenforce
sudo ausearch -m AVC -ts recent
sudo journalctl -u httpd --since '-15 min' --no-pager
```

Use the vhost's real `CustomLog` and `ErrorLog` paths, which may differ from `/var/log/httpd/access_log` and `/var/log/httpd/error_log`. Extract a small window around your teammate's request timestamps. A service journal does not replace HTTP access logs. If `getfacl` or audit tooling is missing, use available metadata/logs and record the limitation.

A permission issue needs a relevant low-privilege principal and an affected website asset. An intentionally writable upload directory is not equivalent to writable application code or an authorization file. SELinux enforcing is useful evidence of a control; it does not establish that every file is correctly protected. File labels and policy denials help explain the observed access. Red Hat Apache and SELinux guidance

Identify group membership using the actual authentication provider. `getent group` describes Linux NSS groups; it does not establish membership in Apache `AuthGroupFile` or an LDAP group. For a file provider, inspect the named group file and necessary username membership without exposing password hashes. For LDAP, inspect its search base/filter, required group DN and subgroup settings. Apache LDAP provider

6. Validate the access matrix from an approved client.

Use the same restricted resource from the same permitted source location for the identity comparisons. Make independent requests or use separate browser sessions so a previous login cannot contaminate the anonymous/non-member result. The valid non-member must have working credentials; a bad-password failure does not demonstrate group enforcement.

Identity	Expected access to the restricted resource	Useful evidence

Anonymous	Restricted content/function unavailable	Challenge, denial or login redirect, with no protected data
Authorized group member	Required content/function available	Actual protected content or issued benign functional check
Valid authenticated non-member	Restricted content/function unavailable	Denial after valid authentication; group/provider evidence

An HTTP 200 can be a login page. A 302 can be a normal authentication redirect. A 401/403 can be caused by something other than the required group rule. Compare response body, headers, identity and server logs. For functionality that changes data, use only the organizer-defined benign validation; reading a page does not prove permission to perform every backend operation.

This Bash example is for a site using HTTP Basic authentication over HTTPS. `192.0.2.10`, `site.example.test` and `/restricted/` are documentation examples, not the missing topology. Supply the organizer's CA bundle. If the correct CA is already trusted, the explicit `--cacert` option may be omitted.

```
WEB_IP='192.0.2.10'
WEB_FQDN='site.example.test'
RESTRICTED='/restricted/'
CA_BUNDLE='/path/to/organizer-ca.pem'
MEMBER='issued-member-username'
NONMEMBER='issued-valid-nonmember-username'

umask 077
mkdir -p evidence
date -u +%FT%TZ
ip -br address
ip route get "$WEB_IP"
getent ahosts "$WEB_FQDN"

# Ordinary DNS path: preserve the result before pinning an address.
curl --connect-timeout 5 --max-time 15 --cacert "$CA_BUNDLE" \
  -sS -D evidence/E04-dns.headers -o evidence/E04-dns.body \
  "https://${WEB_FQDN}${RESTRICTED}"

# The remaining tests pin the issued IP while retaining hostname/SNI.
curl --connect-timeout 5 --max-time 15 --cacert "$CA_BUNDLE" \
  --resolve "${WEB_FQDN}:443:${WEB_IP}" \
  -sS -D evidence/E04-anon.headers -o evidence/E04-anon.body \
  -w 'status=%{http_code} peer=%{remote_ip}\n' \
```

```
  "https://${WEB_FQDN}${RESTRICTED}"

curl --connect-timeout 5 --max-time 15 --cacert "$CA_BUNDLE" \
  --resolve "${WEB_FQDN}:443:${WEB_IP}" --user "$MEMBER" \
  -sS -D evidence/E05-member.headers -o evidence/E05-member.body \
  -w 'status=%{http_code} peer=%{remote_ip}\n' \
  "https://${WEB_FQDN}${RESTRICTED}"

curl --connect-timeout 5 --max-time 15 --cacert "$CA_BUNDLE" \
  --resolve "${WEB_FQDN}:443:${WEB_IP}" --user "$NONMEMBER" \
  -sS -D evidence/E06-nonmember.headers -o evidence/E06-nonmember.body \
  -w 'status=%{http_code} peer=%{remote_ip}\n' \
  "https://${WEB_FQDN}${RESTRICTED}"
```

`--user username` prompts for the password, keeping it out of the command line. Avoid verbose request traces containing credentials. Inspect redirects before following them. For forms, Kerberos/Negotiate or another session mechanism, use the matching installed client or isolated browser sessions; Basic-auth curl commands are not a substitute for those flows.

`--resolve` is useful for isolating DNS from service behavior. Label the result as an address-pinned test; success does not prove DNS works. Preserve normal-path testing too. A mandatory proxy changes the path, so honor the issued proxy design rather than silently bypassing it. curl manual

7. Check transport protection and distinguish it from authentication.

From the approved client, use an unauthenticated HTTP request to the same restricted path. Record whether the server redirects to HTTPS, challenges for credentials over HTTP, or serves restricted material. Do not send real credentials over HTTP to establish that a plaintext challenge exists.

```
curl --connect-timeout 5 --max-time 15 \
  --resolve "${WEB_FQDN}:80:${WEB_IP}" \
  -sS -D evidence/E07-http.headers -o evidence/E07-http.body \
  "http://${WEB_FQDN}${RESTRICTED}"

timeout 15 openssl s_client \
  -connect "${WEB_IP}:443" -servername "$WEB_FQDN" \
  -CAfile "$CA_BUNDLE" -verify_hostname "$WEB_FQDN" \
  -verify_return_error -showcerts </dev/null \
  > evidence/E07-tls.txt 2>&1
```

Confirm certificate identity, chain, dates and successful verification. Use the issued trust anchor; an internal CA is not inherently defective. Plain HTTP being open is not automatically a finding if it safely redirects and does not expose protected content or authentication. Basic authentication

itself does not encrypt credentials, so evaluate the actual transport and configuration together. Apache authentication guidance, OpenSSL s_client

Do not use `curl -k` as passing evidence for TLS. If used for a specifically needed diagnostic, label certificate verification as bypassed and keep the original verification failure. Failed negotiation of an older TLS version can result from the client's own crypto policy; it does not independently prove the server disables that version.

8. Treat package versions and scanner output as leads requiring validation.

For the Apache packages, these local queries can supplement configuration review:

```
rpm -q httpd mod_ssl openssl
sudo rpm -V httpd mod_ssl
dnf --cacheonly updateinfo list --security
```

`rpm -V` differences can be legitimate configuration changes. DNF's cached information may be missing or stale; record its availability and date. Empty cached results do not establish absence of vulnerabilities. Match a suspected CVE to the exact distribution release, package release, vendor advisory and affected conditions. Red Hat backports security fixes, so an older upstream Apache version banner is insufficient evidence of an unpatched flaw. Apply the same release-aware discipline to Rocky's own packages and advisories. DNF reference, Red Hat backporting policy

For A1, local review, ordinary requests and relevant logs are the core tools. Use targeted scanning only if the issued rules permit it. This optional port check targets one approved website IP and its documented ports; change the port list to match the task.

```
nmap -sT -Pn -n -p 80,443 --reason --max-retries 1 \
  --host-timeout 30s -oA evidence/web-ports "$WEB_IP"
```

Avoid interpreting “filtered” as proof of the intended firewall rule: routing problems or another filter can produce the same symptom. Broad subnet scans, exploit suites and blanket `--script vuln` runs do not fit the stated A1 configuration-review task. Nmap options

9. Inspect Windows only when A1 explicitly references it for website authorization.

If Apache uses AD/LDAP and the issued scope names that dependency, use the designated DC or an authorized workstation with the ActiveDirectory module. Substitute the exact group identifier and issued accounts. Otherwise, reserve this section for Day 2 practice.

```
Get-Date -Format o
Import-Module ActiveDirectory

$WebGroup = 'issued-website-group'
$Member = 'issued-member-username'
```

```
$Nonmember = 'issued-valid-nonmember-username'

Get-ADGroup -Identity $WebGroup |
  Select-Object Name, DistinguishedName, GroupCategory, GroupScope

Get-ADGroupMember -Identity $WebGroup -Recursive |
  Select-Object Name, SamAccountName, ObjectClass

Get-ADUser -Identity $Member -Properties Enabled, LockedOut |
  Select-Object SamAccountName, Enabled, LockedOut
Get-ADUser -Identity $Nonmember -Properties Enabled, LockedOut |
  Select-Object SamAccountName, Enabled, LockedOut

Get-ADAccountAuthorizationGroup -Identity $Nonmember |
  Select-Object Name, DistinguishedName
```

The token-group query needs a global catalog. Recursive group membership and a user's security groups are useful evidence, but Apache's LDAP group/subgroup rules determine what it actually accepts. `memberOf` alone can omit nested/primary-group relationships, and `whoami /groups` describes the current session. Confirm the live website behavior with the actual test identities. A missing module or failed directory connection is a limitation to resolve, not proof of an authorization defect. Microsoft group membership, account authorization groups

10. Inspect the firewall only for a permitted website-path question during A1.

The supplied material does not name the product. Use the actual appliance's runtime policy, NAT, routes, states, logs and packet-capture tools. The pfSense commands below are a conditional example, not an assumption about the competition environment.

First write the exact permitted flow: source client/zone → destination before and after NAT → protocol/port → expected allow/deny. Use the same source location for repeat tests. A firewall's own connection test does not reproduce a forwarded client flow.

On pfSense:

```
date -u
ifconfig
netstat -rn
pfctl -sr
pfctl -vvsr
pfctl -sn
pfctl -ss
```

Inspect interface/floating rules, aliases, relevant anchors, NAT and matching states. The GUI configuration and active PF rules should agree. `pfctl -vvsr` gives counters and identifiers for correlation; package rules may live in anchors. Netgate runtime ruleset guidance

Use a narrow capture on the relevant interface and address as seen there. The interface/IP below are documentation examples. Start the capture, have the teammate send one request, then stop it with Ctrl-C after the result or about 15 seconds if the packet limit is not reached.

tcpdump -ni igb1 -c 40 'host 192.0.2.10 and tcp port 443'

Use the actual ingress and egress interfaces separately when needed; NAT may change the address visible on each. Save a narrowly filtered pcap only if needed and permitted by the evidence instructions. Captures prove the traffic path; they do not by themselves establish which application user is authorized. Netgate tcpdump guidance

For each network restriction, pair an expected-success test with an expected-denial test and the relevant policy/log evidence. An allowed TCP handshake does not establish website authorization. A failed connection alone does not establish correct firewall policy. Existing states can survive policy edits; on Day 2 use fresh test connections and inspect states before concluding a new rule is ineffective. Avoid clearing all states as a routine diagnostic.

11. Triage the strongest two findings.

These are investigation priorities, not claims about the unseen environment:

Candidate	Evidence required before selecting it	Practical interpretation
Anonymous or valid non-member reaches restricted content/function	Correct resource, verified identity/session, actual protected result and effective authorization path	Direct failure of the client's group-access requirement
Authentication material or sensitive website files are exposed	Effective serving path and relevant accessible content, with minimal sensitive collection	Can enable misuse of credentials or disclose private information; explain the evidenced scope
Protected access relies on plaintext transport	Reachable HTTP/authentication behavior plus configuration and intended use	An observer on the relevant network path may obtain credentials or content
Low-privilege principal can alter served code or authorization data	Relevant permissions/ACLs, principal capability and affected service role	Can undermine site integrity or access restrictions; distinguish intended upload writes

Suspected unpatched software flaw	Exact vendor package/advisory applicability and required conditions	Valid only after accounting for backports and the actual configuration

Rank confirmed candidates by impact, ability to reach the affected path, required privileges, likelihood and compensating controls. A missing header, disclosed version or unused weak setting should not automatically outrank demonstrated unauthorized website access. If one issue explains multiple affected URLs, report its root cause once unless a second issue is independently evidenced and requires a distinct corrective action.

The supplied template requests both **severity 0–10** and **risk Low/Medium/High/Critical**, but does not require CVSS. Use the organizer's method if one is issued. Otherwise state your method and explain the numeric severity and contextual risk separately. For a formal CVSS score, record the version/vector and calculate it; do not present an intuitive number as CVSS. CVSS severity alone is not a complete site-risk assessment. FIRST CVSS v3.1 specification

Do not manufacture a second vulnerability to fill the table. If only one is confirmed at the selection gate, focus remaining validation on the strongest in-scope candidate and keep any uncertainty explicit.

12. Write recommendations as a small, verifiable change plan.

For each selected root cause, explain the sequence: preserve the affected configuration → make the smallest relevant correction during the authorized implementation stage → check syntax → apply using the service's supported method → repeat authorized and unauthorized tests → roll back if required functionality breaks.

Evidenced root cause	Precise recommendation pattern	Validation after implementation
File-provider population exceeds required group and `Require valid-user` admits it	Keep the intended authentication provider; apply the correct `AuthGroupFile` and `Require group` for the issued group; remove applicable broader alternatives	Member succeeds; valid non-member and anonymous user cannot reach the protected resource
LDAP authorization does not enforce the issued group	Correct the provider-appropriate required group DN and subgroup behavior; preserve necessary directory connectivity	Same identity matrix plus directory/log correlation

Sensitive files are served from a public path	Move authentication/secrets material outside served roots where appropriate; update references and permissions; close applicable serving paths	Sensitive URLs unavailable; required authentication and public site still work
HTTP permits sensitive authentication	Establish valid HTTPS and intended trust; redirect or deny HTTP before sensitive authentication as appropriate	Trusted hostname-valid HTTPS works; HTTP does not solicit credentials or expose protected content
An untrusted principal can alter protected site/configuration files	Reduce ownership/ACL permissions at the affected paths, preserving only required application write locations	Untrusted write capability removed; intended application operations preserved

These are conditional recommendation patterns. Exact directives depend on the demonstrated provider, vhost and request mapping. Do not paste a file-provider group directive into an LDAP or application-session configuration.

In the official Table 1, keep its fields: vulnerability, description, numeric severity, categorical risk, why it is a problem and recommendations. Put evidence IDs and reproduction details in the description or the organizer-approved evidence attachment; do not silently replace the official template with a different report.

Aim for a 150–250 word executive summary within the 500-word maximum. Explain: what you reviewed; the overall assessment and limits; the two confirmed problems in everyday language; who could be affected and how; the first practical actions; and how success will be checked. Avoid asserting data theft, system takeover or compromise unless the evidence supports that claim. Technical filenames, raw commands and detailed directives belong with the findings/evidence.

13. Use this Day 2 bridge for your normal OS/network roles.

The following work is supported by the supplied marking scheme, but the actual A2 task, thresholds, endpoints, product versions and change-request card are still needed. Do not execute it during A1 merely because it appears here.

Day 2 scored work	Primary owner	Peer handoff	Acceptance evidence
Zones, segmentation, routing, default	Teammate	You supply required server endpoints and ports	Intended allow and representative prohibited flow from the specified clients

deny and required allows			
PKI and HTTPS	You; teammate handles appliance certificates when relevant	Agree CA responsibility, certificate identity, DNS name and required path first	Correct issuance/deployment and trusted hostname-valid HTTPS on the intended client
AD/GPO password, account and additional controls	You	Teammate preserves only necessary identity/DNS/time paths	Effective controls on the intended users/computers, including fine-grained policy if present
SSH and SELinux/HTTPS continuity	You	Give teammate the issued SSH port and allowed source scope	Authorized non-root access succeeds; root/restricted user access fails; enforcing MAC and HTTPS remain operational
Controlled Internet/test-site policy and IDS	Teammate	You help generate only the organizer-defined benign test	Required allowed/blocked websites and correctly timestamped IDS alert
OpenVPN, group access and scoped internal reachability	Teammate; you provide the needed identity/group evidence	Provider, authorized group and exact allowed internal service	Authorized tunnel/service succeeds; prohibited identity/service outcome matches the task
Change request, regressions, log and frozen exports	Both, one owner per change	State exact change scope before implementation	Requested outcome plus preserved security boundaries and prior critical services

Begin independent local preparation in parallel, then integrate dependencies. Agree DNS, time, certificate names, authentication group and required flows early. Do not open an unrestricted path while waiting for a precise service list. The workbook awards service continuity, minimum-scope changes and regressions explicitly.

14. Use these broader Rocky/RHEL checks for Day 2 or authorized practice.

```
sudo sshd -t

# Replace user/source/address/port with an actual issued test connection.
```

```
sudo sshd -T -C
user=issuedadmin,host=client.example.test,addr=192.0.2.20,laddr=192.0.2.10,lport=2222 \
  | grep -Ei
'^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|allowgroups|de
nyusers|denygroups) '

sudo ss -lntp
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all-zones
sudo firewall-cmd --permanent --list-all-zones
sudo nft list ruleset
getenforce
sestatus
sudo semanage port -l | grep ssh_port_t
sudo ausearch -m AVC -ts recent
```

Use firewall commands for the installed backend. Compare persistent intent with runtime state. `sshd -T -C` evaluates connection-dependent Match rules for the supplied context, so repeat with relevant allowed/restricted identities. It parses current files; follow with actual access tests. On Rocky/RHEL, a non-default SSH port also needs appropriate SELinux port labeling and firewall policy. Preserve a working management session while making an authorized change, and test a second session before closing it. OpenSSH sshd manual

A broad compliance scan is optional practice/A2 work if it is in scope, permitted and the content is already installed. Discover the matching data stream and exact profile rather than assuming that RHEL content automatically applies to Rocky or that a profile ID exists.

```
# Select an installed data stream that supports the exact OS and release.
find /usr/share/xml/scap/ssg/content -maxdepth 1 -name '*-ds.xml' -print
SCAP_DS='/path/to/matching-installed-datastream.xml'
oscap info "$SCAP_DS"

# Copy an actual supported profile ID from the preceding output.
SCAP_PROFILE='actual-profile-id'
sudo oscap xccdf eval --profile "$SCAP_PROFILE" \
  --results scap-results.xml --report scap-report.html "$SCAP_DS"
```

Use evaluation only; do not add automatic remediation. Examine failed rules for relevance and treat `notapplicable`, missing content and evaluation errors separately. A compliance result is not a complete CVE assessment; a vulnerability evaluation needs suitable, current vendor data. Red Hat OpenSCAP guidance

15. Use these broader Windows checks for Day 2 or authorized practice.

Run elevated where needed on the intended target, with the relevant roles/modules already installed. Capture evidence to the designated working directory. A localized OS may use different names for audit-policy subcategories.

```
New-Item -ItemType Directory -Path C:\Evidence -Force | Out-Null
Get-Date -Format o
Get-NetIPConfiguration
Get-NetConnectionProfile
w32tm /query /status

Get-CimInstance Win32_OperatingSystem |
  Select-Object Caption, Version, BuildNumber, LastBootUpTime
Get-HotFix | Sort-Object InstalledOn -Descending |
  Select-Object -First 10 HotFixID, InstalledOn

Get-ADDefaultDomainPasswordPolicy
Get-ADUserResultantPasswordPolicy -Identity 'issued-test-user'
gpresult /scope computer /h C:\Evidence\gpresult-computer.html /f
auditpol /get /category:*

Get-NetFirewallProfile -PolicyStore ActiveStore |
  Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction

$InboundAllow = Get-NetFirewallRule -PolicyStore ActiveStore |
  Where-Object { $_.Enabled -eq 'True' -and $_.Direction -eq 'Inbound' -and $_.Action -eq 'Allow'
}
$InboundAllow | Select-Object Name, DisplayName, Profile, PolicyStoreSource
$InboundAllow | Get-NetFirewallPortFilter
$InboundAllow | Get-NetFirewallAddressFilter

Get-NetTCPConnection -State Listen |
  Select-Object LocalAddress, LocalPort, OwningProcess

Get-SmbServerConfiguration |
  Select-Object EnableSMB1Protocol, EnableSMB2Protocol, RequireSecuritySignature,
EncryptData

Get-WinEvent -FilterHashtable @{
  LogName = 'Security'
  Id = 4624,4625
  StartTime = (Get-Date).AddMinutes(-15)
} -MaxEvents 30

# Replace with the issued service endpoint; test from the prescribed client.
```

Test-NetConnection -ComputerName 'issued-server-fqdn' -Port 443

Compare the user's resultant fine-grained password policy, when present, with the intended baseline; otherwise assess the applicable domain policy. A GPO existing in the console does not prove application. Check `gpresult` on the target computer and user results in the actual target user's session when user controls are involved. Resultant password policy, gpresult

`ActiveStore` represents the effective combined firewall policy. Correlate rule identities with their port, address, application/service and interface conditions; the port-filter list alone can hide an excessive source scope. A `NotConfigured` profile field requires interpretation of effective defaults, GPO and an actual traffic test. Microsoft firewall profiles, firewall rules

SMB settings are host configuration evidence; verify the protocol/signing/encryption actually required by the task and negotiated by the relevant client. `Get-HotFix` is partial update inventory, not proof that all applicable CVEs are remediated. Match the OS build and installed cumulative updates to suitable offline advisory data when such checking is in scope. Missing log events can reflect audit configuration or test conditions, not necessarily absence of activity. Microsoft SMB server configuration

16. Troubleshoot one layer at a time, then finish the report.

Observation	Next check	Owner
Name fails, address-pinned hostname request works	Issued DNS server, record, suffix and proxy path	Teammate; you if the relevant DNS service is yours
TCP connection fails	Source route/interface, ingress/egress captures, NAT, return path, host listener/firewall	Teammate owns path; you own host state
TCP works, TLS verification fails	Clock, trust bundle, certificate chain, hostname/SNI and endpoint identity	You, with teammate confirming the endpoint
TLS works, valid member cannot authenticate	Correct provider/username form, test credentials, account state, provider connectivity and logs	You
Valid non-member sees protected result	Effective authorization logic, group/subgroup mapping, session isolation and alternate path	You trace root cause; teammate reproduces

Authorized access works but prohibited access also works	Relevant rule scope, matching state or group authorization; verify the intended test source	Owner of the effective control

During A1, stop troubleshooting once you have enough evidence to explain the scoped configuration issue; recommend the fix unless a correction is expressly permitted. During Day 2, make one reversible, minimal change, run the relevant positive/negative tests and then repeat critical service checks.

Before the announced freeze, verify:

- The official A1 template contains exactly two distinct findings and every required field.
- The executive summary is at most 500 words and understandable to a non-technical client.
- Each selected finding has retrievable evidence that supports the precise claim, including actual identity, source location and affected resource.
- Severity/risk reasoning distinguishes observed facts, inferred impact and limitations.
- Recommendations address the root cause, name the responsible layer and state how required service and denial behavior will be verified.
- Evidence files open, references resolve within the submitted pack, and no required evidence exists only in a terminal window.
- The prescribed filename, location and export convention are correct. Use the organizer's convention rather than the example names in this runbook.
- Content is ready before time expires. At the announced finish, stop assessment/changes, perform only the prescribed Freeze & Export actions and leave the files in the required location. Do not edit findings or configurations after freeze.

This runbook was checked against the supplied documents and official command documentation. The commands have not been exercised on the unseen competition VMs; actual versions, installed tools and issued requirements determine applicability.

Tab 3

Your Day 1 strategy should focus on proving two important website vulnerabilities and producing a strong report. The supplied A1 project specifies an assessment of an existing Apache installation. General Windows, routing, and firewall hardening belongs on Day 2.

I prepared a complete, copyable runbook with commands, evidence templates, troubleshooting steps, and separate Day 2 checks:

WorldSkills-A1-Team-Runbook.md

The recommendations below follow your actual documents.

Your A1 PDF and marking scheme establish these boundaries:

- Assess Apache and the website configuration.
- Examine other systems only when explicitly referenced for website access-control validation.
- Submit **exactly two findings** in the official template.
- Keep the executive summary within **500 words**.
- Expect no Internet connectivity.
- Follow the announced Freeze & Export procedure.
- Day 2 is called **Module A2**, under marking subcriterion **B1**.

Neither the actual duration nor the topology is supplied, so the schedule below uses percentages.

For your framework, use **NIST SP 800-115, shortened into six chronological gates: Scope → Baseline → Validate → Prioritize → Report → Freeze.** NIST's configuration, ruleset, and log review techniques closely match this task. This sequence is my competition adaptation, rather than an official WorldSkills framework. Use CIS Controls 4, 6, and 8 as reminders to check configuration, access control, and logging. NIST assessment guidance, CIS Controls

Gate	What you must establish
Scope	Exact website, restricted functions, authorized group, permitted clients, and submission requirements
Baseline	Current service, vhost, authentication provider, addresses, and timestamps
Validate	What configuration allows, and what a real client can actually do
Prioritize	The two strongest distinct findings, accounting for impact and compensating controls
Report	Evidence-backed findings, practical recommendations, and a clear executive summary

Freeze	Complete, readable submission using the issued filename and export convention

For every candidate finding, follow this chain:

Requirement → observed configuration → observed behavior → practical impact → corrective action → verification after correction.

That approach directly supports the scoring:

A1 scoring area	Marks
Submission compliance and scope	2
Evidence supporting both findings	4
Executive-summary clarity and risk communication	4
Prioritization	2
Technical and practical recommendations	3
Total	**15**

Evidence and executive-summary quality account for **8 of 15 marks**. Protect time for writing.

For delegation, keep your OS ownership, but give your teammate responsibility for **client-side validation and evidence coordination** during A1. This creates useful parallel work even when configuring the firewall is outside scope.

Elapsed time	You: OS/Apache	Teammate: clients, path, evidence
0–10%	Confirm scope, access, and report fields	Confirm permitted test clients, submission location, and freeze rules
10–30%	Identify active vhost, includes, document root, and authentication provider	Verify documented connectivity; test anonymous access and HTTP/TLS behavior
30–50%	Trace authorization rules, relevant permissions, and server logs	Test authorized member versus valid non-member; check documented alternate paths
50–60%	Confirm root causes and technically appropriate fixes	Reproduce key observations and check competing explanations

60–80%	Complete the two technical findings	Draft executive summary and assemble evidence index
80–90%	Review summary accuracy	Review whether evidence actually proves each claim
90–100%	Check fields, word count, and final content	Prepare and inspect the prescribed export

For a **120-minute rehearsal**, those checkpoints are minutes **12, 36, 60, 72, 96, 108, and 120**. This is an example, not the official duration.

Use three strict coordination rules:

- **One owner per device and one owner for the report file.**
- Share the FQDN, IP/port, restricted URL, authentication provider, group, and account labels as soon as they are identified.
- After three minutes blocked, send the other person the source IP, destination, port, timestamp, symptom, and evidence ID. Continue an independent task while the responsible person investigates.

Aim to select your findings by **60% elapsed** and complete the report draft by **80%**.

For vulnerability assessment, start with the website requirement and work outward.

First, inspect the Rocky/RHEL Apache configuration. These commands establish the platform, service, virtual hosts, modules, and listeners:

```
date -u +%FT%TZ
cat /etc/os-release
ip -br address
ip route

rpm -q httpd mod_ssl openssl
sudo systemctl status httpd --no-pager
sudo systemctl cat httpd

sudo httpd -V
sudo httpd -t
sudo httpd -S
sudo httpd -M
sudo ss -lntp
```

Check the service unit for a custom configuration path. Inspection commands must use the same configuration arguments as the running service.

`httpd -t` establishes syntax validity; it does not establish correct security behavior. Corroborate configuration on disk with live responses. Apache command documentation

Then locate the relevant controls:

```
sudo rg -n -i \

'Include|VirtualHost|DocumentRoot|Alias|Directory|Location|AuthType|AuthBasicProvider|AuthUserFile|AuthGroupFile|AuthLDAPURL|Require|AuthMerging|AllowOverride|Options' \
  /etc/httpd/conf /etc/httpd/conf.d
```

Use `grep -RniE` if `rg` is unavailable. Read the surrounding blocks and follow active includes.

Pay particular attention to:

- Whether the required group is actually enforced.
- Whether another applicable authorization clause permits access.
- Whether `.htaccess` restrictions are ignored.
- Whether HTTP, aliases, or another documented vhost expose the same restricted resource differently.
- Whether relevant authentication or website files have inappropriate permissions or serving rules.

A suspicious directive is a lead. For example, `Require valid-user` becomes relevant when its provider accepts users outside the required group. Establish the provider population and actual behavior before reporting it. Configuration merging also matters. Apache authentication, configuration sections

Next, prove the access matrix. This should be your teammate's highest-priority test:

Test identity	Expected restricted-resource result
Anonymous	Protected content/function unavailable
Authorized group member	Required content/function available
Valid authenticated non-member	Protected content/function unavailable

The non-member needs **working credentials**. A wrong-password failure does not demonstrate group enforcement.

For HTTP Basic authentication over HTTPS:

```
# Replace these examples with the issued values.
```

```
URL='https://site.example.test/restricted/'
CA='/path/to/organizer-ca.pem'

curl --cacert "$CA" -sS -D anon.headers \
  -o anon.body "$URL"

curl --cacert "$CA" --user 'issued-member' \
  -sS -D member.headers -o member.body "$URL"

curl --cacert "$CA" --user 'issued-valid-nonmember' \
  -sS -D nonmember.headers -o nonmember.body "$URL"
```

Passwords are prompted. Use separate browser sessions or the matching client mechanism if the website uses forms or integrated authentication.

Inspect **headers, response body, and server logs together**. A `200` response may contain a login page. Reading a protected page also does not prove permission to execute every restricted backend operation.

The runbook includes hostname-preserving `--resolve` tests for isolating DNS issues. Label those separately from normal DNS-path results. curl documentation

Check transport protection and relevant host evidence. Verify the certificate against the intended CA and hostname:

```
openssl s_client \
  -connect WEB_IP:443 \
  -servername WEBSITE_FQDN \
  -CAfile /path/to/organizer-ca.pem \
  -verify_hostname WEBSITE_FQDN \
  -verify_return_error </dev/null
```

Substitute the issued values first. Test HTTP without credentials and inspect whether it redirects safely, exposes content, or solicits authentication. `curl -k` cannot serve as passing evidence for certificate validation. OpenSSL documentation

For the actual website paths, use:

```
sudo namei -l /actual/protected/file
sudo getfacl -p /actual/protected/file
sudo ls -lZ /actual/protected/file
getenforce
sudo ausearch -m AVC -ts recent
```

Correlate your requests with the vhost's actual access and error logs. Assess writable paths according to their role: an intended upload directory differs from writable application code or authorization configuration.

Do not conclude that Apache is vulnerable from its version banner alone. RHEL security fixes may be backported into older upstream versions; verify exact vendor package releases and advisory applicability. Missing or stale offline update data is a limitation. Red Hat backporting policy

For Windows, use A1 checks only when AD is an explicitly permitted website dependency.

```
Import-Module ActiveDirectory

Get-ADGroup -Identity 'issued-web-group'

Get-ADGroupMember -Identity 'issued-web-group' -Recursive |
  Select-Object Name, SamAccountName, ObjectClass

Get-ADUser -Identity 'issued-nonmember' -Properties Enabled, LockedOut

Get-ADAccountAuthorizationGroup -Identity 'issued-nonmember'
```

Compare membership with Apache's actual LDAP group logic. Recursive AD membership does not automatically establish how Apache handles nested groups. The authorization-group query requires a global catalog. Microsoft group membership, authorization groups

For **Day 2**, the runbook expands this into effective password policy, GPO results, firewall policy, SMB settings, listening services, and event-log checks. Particularly useful commands include:

```
Get-ADDefaultDomainPasswordPolicy
Get-ADUserResultantPasswordPolicy -Identity 'issued-test-user'
gpresult /scope computer /r
Get-NetFirewallProfile -PolicyStore ActiveStore
```

Evaluate settings on the intended target. A GPO appearing in the management console does not prove it applies. Microsoft gpresult

For the firewall, establish the exact flow before checking policy.

Record source client/zone, destination before and after NAT, protocol/port, and expected outcome. The supplied files do not identify the firewall product.

If it is **pfSense**, relevant inspection commands are:

```
netstat -rn
```

pfctl -sr
pfctl -vvsr
pfctl -sn
pfctl -ss

Pair runtime rules and counters with a client test and narrowly filtered capture. Inspect relevant anchors and NAT as necessary. A test originating on the firewall does not reproduce a forwarded client connection. Netgate ruleset documentation

For every required restriction, demonstrate both:

- The required connection succeeds.
- The prohibited connection fails for the intended control.

A timeout alone cannot prove correct firewall enforcement. Likewise, an allowed TCP connection cannot prove correct website authorization.

Finally, select your two findings by **demonstrated impact, reachability, required privileges, likelihood, and compensating controls**. Prioritize proven unauthorized website access or sensitive exposure over cosmetic configuration issues.

For each recommendation, specify the affected configuration, smallest practical correction, implementation order, and positive/negative verification. The template requests a 0–10 severity and a risk category but does not mandate CVSS. If you use CVSS, calculate and record the version/vector rather than labeling an intuitive number as CVSS.

The runbook's commands were checked against official documentation, and its shell blocks passed syntax checks. They have not been run against the unseen competition VMs.